# Cours — Intégrité et hachage

Nous avons vu les critères **DICP** (Cours 4). On s'attaque maintenant au « comment » technique du critère **Intégrité** : garantir qu'une donnée n'a pas été modifiée. La technique reine est le **hachage**.

## Le besoin : garantir l'intégrité

L'**intégrité** garantit que les données sont **exactes et non altérées** — ni par accident (erreur de transmission), ni par malveillance (attaque de type *man-in-the-middle*). Sans elle, impossible de faire confiance à un document, une mise à jour logicielle ou un ordre de virement.

## Trois familles de mécanismes

| Mécanisme | Rôle |
|---|---|
| **Checksum** (ex. CRC32) | Détecte les altérations **accidentelles**. Rapide, mais **non sécurisé** (ne résiste pas à une attaque). |
| **Hachage cryptographique** (ex. SHA-256) | Détecte les altérations **accidentelles ET malveillantes**. Résistant, irréversible. |
| **Signature numérique** | Ajoute l'**authentification** : prouve **qui** a produit la donnée (cours « RSA : signature et chiffrement hybride »). |



Formule à retenir (conclusion du TD de cette séance) : un **checksum protège de la malchance**, un **hash protège de la malveillance**, une **signature protège de la malveillance et du mensonge** (usurpation d'identité).

## Le hachage : principe

Le **hachage** transforme une entrée (le « message », de taille quelconque) en une chaîne de longueur **fixe** : la **valeur de hachage** ou **empreinte** (*hash*).

C'est une **empreinte numérique** du contenu : si le contenu change, ne serait-ce que d'un caractère, l'empreinte change.

## Les propriétés d'une bonne fonction de hachage

- **Longueur fixe** : quelle que soit la taille de l'entrée, la sortie a toujours la même longueur. SHA-256 produit **256 bits**, soit **32 octets** — qu'on affiche en **64 caractères hexadécimaux**, puisqu'il faut deux chiffres hex par octet.
- **Rapidité** : le calcul est rapide.
- **Résistance aux collisions** : deux entrées différentes ne devraient jamais donner la même empreinte.
- **Irréversibilité** : impossible de retrouver l'entrée à partir de l'empreinte. Si `y = f(x)`, connaissant `y` et `f`, on ne peut pas retrouver `x`.
- **Effet d'avalanche** : une **toute petite** modification de l'entrée change **radicalement** l'empreinte.



L'**irréversibilité** est la propriété clé pour le **stockage des mots de passe** (séance suivante) : si la base fuite, on ne doit pas pouvoir remonter aux mots de passe.

## À quoi ça sert : l'exemple de Bob et Alice

> Bob est en prison et veut envoyer un message à Alice. Il sait que le gardien, Renard, lira le message quoi qu'il arrive — mais Bob veut qu'Alice puisse **détecter** si le message a été **modifié** en chemin.
>
> Bob envoie donc **deux choses** : le **message** et son **empreinte** (hash). À la réception, Alice recalcule l'empreinte du message reçu et la **compare** à celle envoyée par Bob. Identiques → message intact. Différentes → message altéré.

Le hachage ne **cache** pas le message (ce n'est pas du chiffrement) : il permet d'en **vérifier l'intégrité**.

## Attention aux fonctions obsolètes

Toutes les fonctions de hachage ne se valent pas. **MD5**, longtemps utilisée, est aujourd'hui **cassée** : on sait fabriquer des **collisions** (deux fichiers différents avec le même hash MD5). Elle ne doit plus servir à garantir l'intégrité en contexte de sécurité. On utilise **SHA-256** (famille SHA-2) ou plus récent.

## Mémo

- **Hachage** = empreinte de **longueur fixe**, **irréversible**, à **effet d'avalanche**.
- Il **vérifie l'intégrité**, il ne chiffre pas (le message reste lisible).
- **Checksum** (malchance) < **hash cryptographique** (malveillance) < **signature** (malveillance + identité).
- **MD5** = **obsolète** (collisions connues) ; utiliser **SHA-256**.

---